College of Computing

Lab 4: Creational Patterns
Professor : Fahd KALLOUBI
Year : 2026/2027
Major : Computer Science (CS1)

In this Lab, we will put the creational patterns into practice. For each problem you will start from a small Java design that works but creates its objects badly, name the smells, and refactor it toward the right pattern: Abstract Factory for families of related objects, Builder for complex objects, and Prototype for objects copied from presets.

Learning outcomes:

- Recognise the code smells that come from the way objects are created
- Refactor Java code toward Abstract Factory, Builder and Prototype
- Explain the trade-offs of each pattern, such as the cost of adding a new kind of product
- Choose the right creational pattern for an object-creation problem

Work in one Java project, with one package per problem (`problem1`, `problem2`). For every problem, keep the original version in a sub-package (e.g., `problem1before`) and write your refactored version in a sub-package (e.g., `problem1after`), so that both can be compared.

Problem 1:

Abstract Factory. The reporting module of the university must run on several database servers. Each server needs its own connection and its own SQL dialect: MySQL writes `LIMIT n` at the end of a query, whereas SQL Server writes `TOP n` right after `SELECT`. A connection and a dialect must always come from the same family. The current code is the following (create the package `problem1before`).

```
public class MySqlConnection {
    public String connect() { return "Connected to MySQL on port 3306"; }
}
public class MySqlDialect {
    public String selectTop(String table, int n) {
        return "SELECT * FROM " + table + " LIMIT " + n;
    }}
public class SqlServerConnection {
    public String connect() {
        return "Connected to SQL Server on port 1433";
    }}
public class SqlServerDialect {
    public String selectTop(String table, int n) {
        return "SELECT TOP " + n + " * FROM " + table;
    }}
public class ReportDao {
    private final String db;
    public ReportDao(String db) { this.db = db; }

    public void topStudents(int n) {
        if (db.equals("mysql")) {
            MySqlConnection c = new MySqlConnection();
            System.out.println(c.connect());
            System.out.println(new MySqlDialect().selectTop("students", n));
        } else if (db.equals("sqlserver")) {
            SqlServerConnection c = new SqlServerConnection();
            System.out.println(c.connect());
            System.out.println(new MySqlDialect().selectTop("students", n));
        } else {
            throw new IllegalArgumentException("Unknown database: " + db);
        }}}
```

1. In a `Test` class, call `new ReportDao("mysql").topStudents(3)` and `new ReportDao("sqlserver").topStudents(3)`. Compare the two outputs. What is wrong, and what would a real SQL Server answer to this query? Why did the compiler not detect the problem? (Hint: SQL Server does not understand the keyword LIMIT)
2. The team plans to add `countStudents()` and `lateStudents()` to `ReportDao`, each with the same `if/else`. List the code smells of this design and the principle at stake for each one.
3. In the package `problem1after`, declare two product interfaces: `DbConnection` with `String connect()` and `SqlDialect` with `String selectTop(String table, int n)`. Make the four existing classes implement them.
4. Declare the abstract factory `DatabaseFactory` with the methods `DbConnection createConnection()` and `SqlDialect createDialect()`, and implement it in `MySqlFactory` and `SqlServerFactory`.
5. Rewrite `ReportDao` so that it receives a `DatabaseFactory` through its constructor. The class must contain no `if`, no `switch` and no `new` on a concrete product. Explain why the bug of question 1 can no longer happen.
6. Add an Oracle family: Oracle listens on port 1521 and writes `SELECT * FROM students FETCH FIRST 3 ROWS ONLY`. List the files you created and the existing files you had to edit.
7. The family must now be chosen from a configuration string (`"mysql"`, `"sqlserver"` or `"oracle"`). Add a static method `DatabaseFactory forName(String name)` to the interface, and use it in the `Test` class to produce the output below. This method still contains a `switch`: why is it acceptable here, whereas the `if/else` of `ReportDao` was not?
8. The team now needs a third kind of product, a `SqlEscaper` that quotes a column name (between backticks for MySQL, square brackets for SQL Server and double quotes for Oracle). How many existing files must be edited? Which trade-off of the Abstract Factory pattern does this illustrate?
9. Compare with the Factory Method of the lecture (`Logistics.createTransport()`). Why would a single factory method `createConnection()` overridden in subclasses of `ReportDao` not be enough to solve the problem of question 1?

Expected Output (question 7):

```
Connected to MySQL on port 3306
SELECT * FROM students LIMIT 3
Connected to SQL Server on port 1433
SELECT TOP 3 * FROM students
Connected to Oracle on port 1521
SELECT * FROM students FETCH FIRST 3 ROWS ONLY
```

Problem 2:

Builder and Prototype. An online store sells laptops that customers configure before ordering. The `Laptop` class and one of its clients currently look like this (create the package `problem2before`).

```java
public class Laptop {
    public String cpu;
    public int ramGb, ssdGb;
    public boolean gpu, backlitKeyboard;
    public List<String> accessories = new ArrayList<>();

    public Laptop(String cpu, int ramGb, int ssdGb,
                  boolean gpu, boolean backlitKeyboard) {
        this.cpu = cpu; this.ramGb = ramGb; this.ssdGb = ssdGb;
        this.gpu = gpu; this.backlitKeyboard = backlitKeyboard;
    }
    public Laptop(String cpu, int ramGb) {
        this(cpu, ramGb, 256, false, false);
    }
    public Laptop(String cpu, int ramGb, int ssdGb) {
        this(cpu, ramGb, ssdGb, false, false);
    }
    // ... one more constructor per useful combination
}

// Client
Laptop a = new Laptop("i7", 16, 512, false, true);
Laptop b = new Laptop("i5", -8);      // accepted
b.ramGb = 64;                         // changed after the order was placed
```

Part A: Builder

1. Without reading the constructor, what do `false` and `true` mean in the first line of the client? Name the code smells of this class.
2. The store wants one more constructor that sets the size of the SSD and keeps the default RAM: `Laptop(String cpu, int ssdGb)`. Why does it not compile?
3. In the package `problem2after`, rewrite `Laptop` with a nested static class `Builder`:
    a. the CPU is required: it is the only parameter of the `Builder` constructor
    b. the optional steps are `ram(int)` (default 8), `ssd(int)` (default 256), `gpu()`, `backlitKeyboard()` and `accessory(String)`; each one returns the builder
    c. `build()` checks that the RAM is between 4 and 64 GB and that the SSD has at least 128 GB; otherwise it throws an `IllegalStateException` with a clear message
    d. `Laptop` has a private constructor and only `private final` fields; its list of accessories cannot be modified from outside (use `List.copyOf`)
4. Override `toString`, then build an i7 laptop with 16 GB of RAM, a 512 GB SSD, a backlit keyboard and a mouse, and print it. Then try to build an i5 laptop with -8 GB of RAM and print the message of the exception (first two lines of the output below).

Part B: Prototype

5. The sales staff prepare quotes from presets. A `Quote` holds the name of a customer, a `Laptop` and a list of services. Write the class `Quote` with a constructor `Quote(Laptop laptop)`, the methods `Quote forCustomer(String name)` and `Quote addService(String service)` (both return `this`), `Laptop getLaptop()`, `toString`, and a method `Quote copy()` based on a copy constructor. In this first version, the copy constructor copies the list as is: `this.services = src.services;`
6. Write a class `PresetRegistry` with `void register(String name, Quote preset)` and `Quote create(String name)`, which returns a copy of the preset (or throws an `IllegalArgumentException` for an unknown name). Register a preset `"student"`: an i5 laptop with 8 GB of RAM and the service `Setup`. Create two quotes from it, one for SAFI Amal with the service `Extended warranty`, and one for ALAMI Samir. Print

both quotes. What do you observe? Explain it with the difference between shallow and deep copy.

7. Fix `copy()` so that the program prints the output below; the last line prints `q1.getLaptop() == q2.getLaptop()`. Why can the two copies share the same `Laptop` object without any risk, whereas they cannot share the list of services?
8. Should `PresetRegistry` be a Singleton? Give one argument for and one argument against.
9. For each situation below, name the creational pattern you would use (Singleton, Factory Method, Abstract Factory, Builder or Prototype) and justify your choice in one sentence:
    a. an HTTP request has a URL and up to ten optional headers, timeouts and retry settings
    b. a drawing editor duplicates the selected shape without knowing its concrete class
    c. the whole application must share one pool of database connections
    d. a framework lets each kind of application decide which `Document` it creates
    e. a user interface switches between a desktop and a mobile set of widgets that must never be mixed

Expected Output (questions 4 and 7):

```
Laptop[cpu=i7, ram=16GB, ssd=512GB, gpu=no, backlit=yes, accessories=[Mouse]]
Invalid laptop: RAM must be between 4 and 64 GB
Quote for SAFI Amal: services=[Setup, Extended warranty]
Quote for ALAMI Samir: services=[Setup]
true
```

Deliverables: the Java project (one package per problem) and your answers to the written questions.